

Discrete Choice Model Estimation with R

Dan Yavorsky

Table of contents

Welcome	9
How to read this book	10
Intended Audience	11
Acknowledgments	12
Colophon	13
 I Foundations	 14
1 Choice Models and Simulation	15
1.1 A Framework for Choice	15
1.2 A Simple Example	16
1.3 A Binary Logit Model Example	17
1.4 Key Learnings	19
 2 Drawing from Densities	 20
2.1 Why We Need Draws	20
2.2 Pseudo-Random Numbers from the Uniform Distribution	20
2.3 The Inverse Transform Method	20
2.3.1 Example: Exponential	20
2.3.2 Example: Gumbel	21
2.3.3 Example: Logistic	22
2.3.4 Example: Normal	23
2.4 Accept-Reject Methods	24
2.5 Transforming Draws	24
2.6 The Multivariate Normal	24
2.6.1 Example: Correlated Normal Draws	24
2.7 Truncated Distributions	24
2.8 Quasi-Random Draws	24
2.9 Key Learnings	24
 3 Simulating from a Model	 25
3.1 Why Simulate?	25
3.2 The Anatomy of a Simulation Script	25
3.3 Example: Simulating Binary Logit Choices	25

3.4	Example: Simulating Binary Probit Choices	25
3.5	From One Choice to a Dataset	25
3.6	Sanity Checks on Simulated Data	25
3.7	The Simulate-and-Recover Workflow	25
3.8	Key Learnings	25
4	Organizing Choice Data	26
4.1	The Shape of Choice Data	26
4.2	Wide and Long Formats	26
4.3	Alternative-Specific and Individual-Specific Variables	26
4.4	Coding Categorical Attributes	26
4.5	The Outside Option	26
4.6	From Data Frame to Likelihood Inputs	26
4.7	Package Conventions	26
4.8	Key Learnings	26
5	From Choice Data to Likelihood	27
5.1	The Likelihood Function	27
5.2	Independence and the Product Form	27
5.3	The Log-Likelihood	27
5.4	Example: The Binary Logit Log-Likelihood in R	27
5.5	Maximum Likelihood Estimation	27
5.6	The Score, the Hessian, and the Information Matrix	27
5.7	A First Estimator: Grid Search	27
5.8	Why Grid Search Fails at Scale	27
5.9	Key Learnings	27
6	Numerical Optimization	28
6.1	Climbing a Hill You Cannot See	28
6.2	Newton-Raphson	28
6.3	Quasi-Newton Methods: BHHH and BFGS	28
6.4	Numerical Derivatives	28
6.5	R's Optimizers	28
6.6	Standard Errors from the Hessian	28
6.7	Practical Realities	28
6.8	End-to-End: Simulate, Estimate, Recover	28
6.9	Key Learnings	28
II	The MNL Family, by Likelihood	29
7	The Multinomial Logit Model	30
7.1	From Two Alternatives to Many	30

7.2	Random Utility and the Extreme Value Distribution	30
7.3	The MNL Choice Probability	30
7.4	Properties and Implications	30
7.4.1	Independence from Irrelevant Alternatives	30
7.4.2	Substitution Patterns and Elasticities	30
7.5	Identification	30
7.5.1	Only Utility Differences Matter	30
7.5.2	The Scale of Utility	30
7.6	Specification Choices	30
7.7	Simulating MNL Data	30
7.8	Key Learnings	30
8	Estimating the MNL by Maximum Likelihood	31
8.1	The MNL Log-Likelihood	31
8.2	Coding <code>loglik_mnl()</code>	31
8.3	Numerical Stability: The Log-Sum-Exp Trick	31
8.4	The Analytic Gradient	31
8.5	Estimation with <code>optim()</code>	31
8.6	Assessing Fit	31
8.7	Validation Against <code>mlogit</code>	31
8.8	A Full Simulate-and-Recover Study	31
8.9	Key Learnings	31
9	Code Performance	32
9.1	Why Speed Matters Now	32
9.2	Measure First	32
9.3	Vectorizing the MNL Log-Likelihood	32
9.4	Precompute What Doesn't Change	32
9.5	Memory Layout and Copies	32
9.6	When R Isn't Enough	32
9.7	Key Learnings	32
10	Relaxing IIA: Nested Logit and GEV Models	33
10.1	When Proportional Substitution Fails	33
10.2	The Nested Logit Model	33
10.2.1	Nests and the Dissimilarity Parameter	33
10.2.2	Choice Probabilities: Marginal times Conditional	33
10.3	Simulating Nested Logit Data	33
10.4	Estimating the Nested Logit	33
10.5	Validation Against <code>mlogit</code>	33
10.6	The GEV Family	33
10.7	Key Learnings	33

11 The Latent Class MNL Model	34
11.1 Heterogeneity Enters	34
11.2 The Latent Class Model	34
11.3 The Likelihood: A Panel Subtlety	34
11.4 Simulating Latent Class Data	34
11.5 Direct Maximization	34
11.6 The EM Algorithm	34
11.7 How Many Classes?	34
11.8 Individual-Level Output	34
11.9 Validation Against <code>gmn1</code>	34
11.10 Key Learnings	34
12 Maximum Simulated Likelihood	35
12.1 Choice Probabilities Without Closed Forms	35
12.2 The Simulated Choice Probability	35
12.3 The Simulated Log-Likelihood and Its Bias	35
12.4 Practical Rules of MSL	35
12.5 Better Draws: Halton and Antithetics	35
12.6 A First MSL Estimation	35
12.7 Relatives: MSM and MSS	35
12.8 Key Learnings	35
13 The Mixed MNL Model by MSL	36
13.1 Random Coefficients	36
13.2 Choosing Mixing Distributions	36
13.3 The Mixed MNL Likelihood	36
13.4 Simulating Mixed MNL Data	36
13.5 Coding the Simulated Log-Likelihood	36
13.6 Estimation	36
13.7 Feeling the Strain	36
13.8 Validation Against <code>mlogit</code> , <code>gmn1</code> , and <code>mix1</code>	36
13.9 Key Learnings	36
14 The Limits of Likelihood	37
14.1 What We Actually Want	37
14.2 Why Not Estimate Each Person's Beta by MLE?	37
14.3 Conditioning on Choices: Individual Estimates After MSL	37
14.4 ... But It Creaks	37
14.5 The Bayesian Road	37
14.6 Key Learnings	37

III The MNL Family, by Bayes	38
15 Bayesian Estimation of the MNL	39
15.1 The Bayesian Mindset	39
15.2 Priors	39
15.3 The Posterior Has No Closed Form Either	39
15.4 Summarizing Distributions with Draws	39
15.5 The Metropolis-Hastings Algorithm	39
15.5.1 The Idea	39
15.5.2 A Toy Target First	39
15.5.3 Why It Works	39
15.6 M-H for the MNL	39
15.7 MCMC Diagnostics	39
15.8 Bayes Meets MLE	39
15.9 Key Learnings	39
16 Gibbs Sampling and Hierarchical Models	40
16.1 Sampling by Blocks	40
16.1.1 A Toy Example: The Bivariate Normal	40
16.2 Conjugacy	40
16.3 The Hierarchical Model	40
16.3.1 Shrinkage: Borrowing Strength	40
16.3.2 Gibbs for the Hierarchical Linear Model	40
16.4 Data Augmentation	40
16.5 Metropolis-within-Gibbs	40
16.6 Key Learnings	40
17 The Hierarchical Bayesian MNL	41
17.1 The Model	41
17.2 The Sampler	41
17.3 Coding the HB-MNL Sampler	41
17.4 Running It	41
17.5 The Payoff: Individual-Level Posteriors	41
17.6 HB and MSL on the Same Data	41
17.7 Validation Against <code>bayesm</code>	41
17.8 Practical Notes	41
17.9 Key Learnings	41
IV Beyond Logit: The Multinomial Probit	42
18 The Multinomial Probit Model	43
18.1 Why Probit?	43

18.2	The Model	43
18.3	Identification	43
18.4	Binary Probit as a Warm-up	43
18.5	Simulating MNP Data	43
18.6	The Estimation Problem Ahead	43
18.7	Key Learnings	43
19	Estimating the MNP	44
19.1	Route 1: Simulated Likelihood	44
19.1.1	The Accept-Reject Simulator and Its Flaws	44
19.1.2	The GHK Simulator	44
19.1.3	MSL with GHK	44
19.2	Route 2: Bayesian Data Augmentation	44
19.2.1	Latent Utilities as Parameters	44
19.2.2	The McCulloch-Rossi Gibbs Sampler	44
19.2.3	Identification Inside the Sampler	44
19.3	The Two Routes Compared	44
19.4	Validation Against <code>bayesm</code>	44
19.5	Key Learnings	44
20	The Hierarchical Bayesian MNP	45
20.1	The Model	45
20.2	The Sampler	45
20.3	Coding the HB-MNP Sampler	45
20.4	HB-MNL or HB-MNP?	45
20.5	Key Learnings	45
V	Using and Extending the Models	46
21	Post-Estimation: Inference, WTP, and Market Simulation	47
21.1	Estimates Are Not Answers	47
21.2	Inference on Functions of Parameters	47
21.3	Willingness-to-Pay	47
21.4	Attribute Importance	47
21.5	Market Simulation	47
21.6	Elasticities	47
21.7	Using Individual-Level Estimates	47
21.8	Key Learnings	47
22	Extensions: Building Your Own Model	48
22.1	The Recipe	48

22.2	Extension 1: The No-Choice Option and Dual Response	48
22.2.1	Modeling “None”	48
22.2.2	Dual-Response Designs and Their Likelihoods	48
22.2.3	What Changes in the Code	48
22.3	Extension 2: A Budget Constraint	48
22.3.1	The Model	48
22.3.2	What Changes in the Code	48
22.4	Extension 3: Flexible Mixing Distributions	48
22.4.1	The Logit-Mixed Logit Model	48
22.4.2	What Changes in the Code	48
22.5	Where to Go from Here	48
22.6	Key Learnings	48
References		49
Appendices		50
A	Appendix	50
A.1	R Packages for Discrete Choice	50
A.2	Notation Glossary	50
A.3	Common Numerical Errors and Their Symptoms	50

Welcome

Warning

This is an early draft. The book is incomplete. Every aspect of it is subject to change. Many things might be incorrect. Are you sure you want to read this yet?

You don't understand it, until you have coded it.

It's a mantra that my mentors instilled in me in graduate school, and one that I propagate on my students today. I believe it in deeply. If you want to understand a statistical model, it is insufficient to interact with the model only through pen and paper. No matter how well you organize the subscripts or how masterfully you interweave elements of the Greek and Roman alphabets, you won't really "get it." To deeply understand a statistical model, you need to be able to simulate data from the model, and then take that simulated data to an estimation routine that enables you to recover the parameters of interest.

This is something you learn to do in graduate school. But, unless you have one of the most caring and pedagogical advisers, no one teaches it to you! Instead, it's a skill that students develop independently and inefficiently between classes and assignment due dates or, in my case, after I was knee-deep in my dissertation research. There is a tremendous imbalance between how immensely important this skill is and the time and instruction dedicated to it.¹ That imbalance is the motivation for this book.

I address this gap in the specific domain of Discrete Choice Modeling. This book stands on its own, but it did not develop in a vacuum: Kenneth Train's masterful text *Discrete Choice Methods with Simulation* (Train 2009), freely available online at <https://eml.berkeley.edu/books/choice2.html>, and the underappreciated tome *Applied Choice Analysis* by David Hensher, John Rose, and William Greene (Hensher, Rose, and Greene 2015) both shaped my thinking, and I cite them (and others) regularly for readers who want deeper coverage of the models' development, properties, and applications.

Train and I align in our thinking:

¹In this context, a *closed-form expression* means a way of writing the integral so that the anti-derivative sign is not part of solution. For example, the integral $\int x \, dx$ has the closed for expression $x^2/2$ plus some constant. We will see later that the Extreme Value distribution is often chosen for f predominantly because it leads to a closed for expression for the choice probability $p(y|x)$.

“...the true value of [] choice modeling is the ability to create tailor-made models. The computation and programming steps that are needed to implement a new model are usually not difficult. An important goal [] is to teach these skills as an integral part of the exposition of the models themselves.”²

However, what is “usually not difficult” for an experienced researcher can be immensely difficult for a typical graduate student. The aim of this book is to ease that difficulty. I demonstrate R code alongside the math. Every model in this book is developed from its mathematical statement all the way into working R code. No mysteries will remain. To borrow a quote from Quintilian, Cobbett, Cooke, and many others: my goal is to take you through the process of programming the simulation and estimation of common discrete choice models *not so that you can understand, but so that you cannot possibly misunderstand*.

How to read this book

This book tells one long story, and it is best read in order because the chapters build on one another — the code especially. In Part I, we assemble a foundation of computational tools: drawing from densities, simulating data from a model and recovering its parameters, organizing choice data, constructing likelihoods, and maximizing them numerically. In Part II, we put those tools to work on the multinomial logit (MNL) model and its likelihood-estimated extensions, climbing until we reach the limits of what maximum likelihood can deliver. Part III starts the climb again from the Bayesian side, building from the Metropolis-Hastings algorithm up to the hierarchical Bayesian MNL — the workhorse model of modern conjoint analysis and the centerpiece of this book. Part IV leaves the logit family for the multinomial probit, where both the classical and Bayesian toolkits get a capstone workout on a model with no closed-form choice probabilities. Part V turns to what comes after estimation — inference, willingness-to-pay, market simulation — and, finally, to extending these models into new, custom, tailored situations for your own research.

When I introduce a model, I provide enough context to motivate and specify it, but my emphasis remains squarely on estimation. When you want a more thorough treatment of a model’s properties, history, or econometric subtleties, follow the citations; Train (2009) and Hensher, Rose, and Greene (2015) in particular reward the effort.

In addition, I strongly encourage you write or copy the code as you work through this book. I would mandate this if I could, but I’m not sitting there next to you and so I must settle for simply sharing my encouragement. Write the code. Play with the code. Break the code. Make it your own. That behavior is where deep understanding comes from, not from highlighting the occasional sentence or equation you find important.

²More precisely, $p(y = 2|x) = \Pr(x + \varepsilon > 1|x = 0.5) = \Pr(\varepsilon > 0.5) = \int_{0.5}^1 f(\varepsilon) d\varepsilon = (0.5\varepsilon)|_{0.5}^1 = 0.25$.

Intended Audience

This book is intended for a narrow audience, predominantly current or former graduate students with an interest in discrete choice modelling who will find value from seeing and interacting with the programmatic implementation of the multinomial logit and its extended family of related models. In other words, someone who read Train (2009) and thought *how would I code that?* Or someone who has run `mlogit` in R or `nlogit` in Stata and wants to “see under the hood” of those estimation routines.

We will simulate data from the statistical models and then estimate the parameters of those models from the simulated data. Doing so will require us to generate random draws from distributions, approximate integrals with monte carlo simulation, maximize likelihood functions with optimization routines, and (when taking a Bayesian approach) explore posterior distributions with Markov Chain Monte Carlo methods.

We will do all of this in R, a freely available software environment for statistical computing and graphics. As a result, I assume you are reasonably familiar with R. If not, there is a tremendous set of free online R resources collected and organized at <https://www.bigbookofr.com/>. Popular books include *R for Data Science* properly cited as Wickham, Cetinka-Rundel, and Grolemund (2023) and *Advanced R* properly cited as Wickham (2019). If you are new to R or your familiarity with it has waned, you should consult these resources before proceeding here.

I also assume you have taken introductory statistics or econometrics courses, as the concepts and techniques taught there are foundational for understanding and estimating discrete choice models. In particular, you should have no uncertainty about the difference between a model, an estimator, and an estimate. To briefly review:

- a *model* is the set of mathematical assumptions about how data are generated,
- an *estimator* (or equivalently, the estimation routine) is an algorithm or function of the data, and
- an *estimate* is the result of applying the estimator to a particular dataset.

In my experience, students are often given a model and an estimator, after which much time in the classroom is spent deriving properties of the estimator for that particular model. Often a homework assignment follows which asks students to implement the estimator on a dataset to find an estimate. While this approach succeeds in developing the student’s skills and understanding of the particular estimator for the particular model, it does so with blinders on, obscuring consideration about the choice or specification of the model as well as the choice of estimator.

I’m sure you can recall your introduction to linear regression where you were handed a model with a linear conditional mean and a Normally distributed “error” and told that you would estimate the parameters of the model via least squares. Hopefully this left you wondering

about less-particular situations such as those with a non-continuous outcome variable, a non-Normally distributed error term, a non-linear conditional mean, correlation between the observed covariates and the error, a causal interpretation, or a different estimation approach.

In this book, we will focus on models with non-continuous outcome variables and we will use Maximum Likelihood and Bayesian methods to estimate the parameters of those models. If you would like to review key ideas from statistics and econometrics in preparation for tackling our discrete choice models with likelihood and Bayesian approaches, I highly recommend Abramovich and Ritov (2023, [link](#)) on the topic of mathematical statistics, Goldberger (1991, [link](#)) and Kennedy (2008, [link](#)) for econometrics, and McElreath (2018, [link](#)) and Gelman et al. (2013, [link](#)) for Bayesian statistics. Bruce Hansen’s recently released two-volume series Hansen (2022b, [link](#)) and Hansen (2022a, [link](#)) on probability, statistics, and econometrics are an excellent set of alternative resources.

Acknowledgments

I gratefully acknowledge the academic giants who pioneered and developed the field of Discrete Choice Analysis. The models offer flexible forms to study human behavior, the estimation methods are fascinating, their implementation usually proceeds from vexing to fun, and their application has been immensely useful in my consulting practice where I seek to help firms better understand consumers in order to improve consumer well-being and firm profitability.

I am also immensely grateful to the teams that work on the [R-Project](#), those at [Posit](#) who provide [RStudio](#), [Positron](#), and [Quarto](#), and the related communities of developers, academics, and R users. The free tools they provide are exceptional and I use them daily. The welcoming communities they have established provide the necessary support humans need, be it technical troubleshooting or emotional encouragement.

More personally, I would like to thank my academic mentors [Ella Honka](#), [Peter Rossi](#), and [Eric Bradlow](#), as well as the folks that have or continue to tolerate me through my consulting work, including [Prachi Bhalerao](#), [June Wu](#), [Chris Diener](#), [Keith Chrzan](#), and the hosts and attendees of [Sawtooth Software](#)’s annual Analytics and Insights Summit. I have learned so much from these researcher’s guidance and often their work.

Special thanks go to generous people who reviewed and assisted with drafts of this book, including [Darren Aeillo](#), [Kalyan Rallabandi](#), and [Geoff Zheng](#).

Unquestionably, my deepest thanks go to Alison, Zach, and Ben for their patience and support.

Colophon

An online version of this book is available at <https://dcme-r.danyavorsky.com>. The source of the book is available at <https://github.com/dyavorsky/dcme-r>. The book is authored using [Quarto](#), an open-source scientific and technical publishing system that makes it easy to create articles, presentations, websites, books, and other publications that combine text and executable code.

Part I

Foundations

1 Choice Models and Simulation

It is instructive to start with the first sentence of Train (2009) Section 1.3, “Discrete choice analysis consists of two interrelated tasks: **specification** of the behavioral model and **estimation** of the parameters of that model” (emphasis added). Notably, chapters 1–7 of Train (2009) only specify models; there is not even a hint of model estimation. We too begin with a focus on model specification.

Let me be clear: many students will see y and x later in this section and immediately think about “fitting” a model; that is, they assume they have data that they will put into an estimation routine to find estimates of the parameters of the model. We are not there yet! At this early stage in the book, we are only specifying models; that is, listing sets of assumptions about data generating processes and exploring the implications of those assumptions. There are no data yet. There will be parameters introduced in our choice of model specification, but we are not yet estimating those parameters. Have patience, we will eventually do these things.

1.1 A Framework for Choice

Train denotes the outcome in any given choice situation as y , determined by some observable factors collected in the vector $\mathbf{x}$ and some unobservable factors collected in the vector $\boldsymbol{\varepsilon}$. The factors ($\mathbf{x}$ and $\boldsymbol{\varepsilon}$) relate to the agent’s choice (y) through a function $y = h(\mathbf{x}, \boldsymbol{\varepsilon})$. We assume for the moment that we know $h(\cdot)$ and that $\mathbf{x}$ and $\boldsymbol{\varepsilon}$ are length-one vectors (i.e., scalars) denoted x and ε .

Since we do not observe ε , we can’t predict y exactly. Instead, we focus on the probability of y , that is:

$$\begin{aligned} p(y|x) &= \Pr(\varepsilon \text{ such that } h(x, \varepsilon) = y) \\ &= \Pr(I[h(x, \varepsilon) = y] = 1) \\ &= \int I[h(x, \varepsilon) = y] f(\varepsilon) d\varepsilon \end{aligned} \tag{1.1}$$

For certain special choices of h and f , a closed-form expression¹ for the integral is available. But more generally, for almost any choice of h and f , we can approximate the integral through simulation. Train provides pseudo code on how to do so:

1. Repeat the following two steps many ($r = 1, \dots, R$) times:
 - Draw $\varepsilon^{(r)}$ from $f(\varepsilon)$.
 - Determine whether $h(x, \varepsilon^{(r)}) = y$. If so, set $I^{(r)} = 1$; else set $I^{(r)} = 0$.
2. Average the R values of I

Next we look at two examples where we use this procedure to approximate the $p(y|x)$ integral. The first example I made up. The second example is the binary logit model discussed by Train, but for which I'll show you how to simulate the $p(y|x)$ integral.

1.2 A Simple Example

Let's first set up a toy example to demonstrate how simulation can approximate the $p(y|x)$ integral. Suppose $x = 0.5$ and ε is uniformly distributed between -1 and 1. Define $h(x, \varepsilon)$ to be:

$$h(x, \varepsilon) = \begin{cases} 0 & \text{if } x + \varepsilon < 0 \\ 1 & \text{if } x + \varepsilon \in [0, 1] \\ 2 & \text{if } x + \varepsilon > 1 \end{cases} \quad (1.2)$$

We'll focus on the outcome $y = 2$. You can probably intuit that the $p(y = 2|x) = 0.25$ since only one quarter of the time will ε be sufficiently positive to make $x + \varepsilon > 1$.² Nevertheless, let's approximate the integral representation of $p(y = 2|x)$ through simulation to ensure we understand the process.

To walk you through the code, we first set a seed so that the pseudo-random numbers generated by `runif()` can be replicated exactly each time the code is run (even on different computers). We then specify that we will use 1,000 draws in the simulation and we create a vector `I` to hold our results. The simulation occurs via a `for()` loop where each time through the loop we take a draw of ε , calculate $0.5 + \varepsilon$ and check whether that sum is greater than one. If so, then $h(x, \varepsilon) = 2$ matching the value of y for the choice probability we want to assess — i.e., $p(y = 2|x)$ — and thus we store a 1 in the r^{th} position of `I`; otherwise we store a 0. We then average the values in `I` to get our approximation of $p(y = 2|x)$.

¹In this context, a *closed-form expression* means a way of writing the integral so that the anti-derivative sign is not part of solution. For example, the integral $\int x dx$ has the closed for expression $x^2/2$ plus some constant. We will see later that the Extreme Value distribution is often chosen for f predominantly because it leads to a closed for expression for the choice probability $p(y|x)$.

²More precisely, $p(y = 2|x) = \Pr(x + \varepsilon > 1|x = 0.5) = \Pr(\varepsilon > 0.5) = \int_{0.5}^1 f(\varepsilon) d\varepsilon = (0.5\varepsilon)|_{0.5}^1 = 0.25$.

```

set.seed(1234)

R <- 1000
I <- vector(length=R)

for(r in 1:R) {
  eps <- runif(1, min=-1, max=1)
  h <- 0.5 + eps
  I[r] <- as.integer(h > 1)
}
mean(I)

```

```
[1] 0.258
```

The simulated value 0.258 approximates the exact value 0.25 and can be made closer by increasing the number of draws used in the simulation.

Notice that we took draws of ε to empirically approximate the integral of $p(y|x)$, but we should not think of these draws as “data” in the sense of a dataset. They are ancillary numbers generated from the distribution of ε that we use to approximate the integral.

R users will recognize that we can shorten the code by taking advantage of R’s vectorized functions and its conversion of boolean values to 0/1 when used in mathematical operations. Here is a shorter implementation of the simulation; whether it’s “better” code is a matter of preference.

```

set.seed(1234)
R <- 1000
mean( runif(R, min=-1, max=1) + 0.5 > 1 )

```

```
[1] 0.258
```

That’s it. If you can generate pseudo-random draws from the density f and you know h , approximating a choice probability by simulation might only require a handful of lines of code.

1.3 A Binary Logit Model Example

As an example of a model with a complete closed-form solution, Train provides the binary logit model. In this example, we’ll let $\mathbf{x}$ and $\boldsymbol{\varepsilon}$ be vectors of length two, and we’ll specify h and f as follows:

The “binary” part refers to the aspect of the model whereby the decision maker does one of two things; they either take an action ($y = 1$) or not ($y = 0$). To tie this model into a framework of behavior, we start with a utility function U . In Train’s specific example, utility is specified as

$$U(\mathbf{x}, \boldsymbol{\beta}, \varepsilon) = \mathbf{x}'\boldsymbol{\beta} + \varepsilon \quad (1.3)$$

where $\mathbf{x}$ is a vector of observable explanatory variables, $\boldsymbol{\beta}$ is a vector of parameters that through the functional form $\mathbf{x}'\boldsymbol{\beta}$ effectively serve as weights on the covariates, and ε is a scalar index collecting the value of information used by the decision maker but unobserved to the researcher. Notice that we’re allowing $\mathbf{x}$ and $\boldsymbol{\beta}$ to be vectors in this example, whereas they were scalars (or equivalently length-one vectors) in the previous example.

In this model, the threshold for action is 0 because the decision maker takes action ($y = 1$) when utility is positive; conversely, when utility is negative, the decision maker elects not to take the action (i.e., $y = 0$). Therefore we can specify h as:

$$h(\mathbf{x}, \boldsymbol{\beta}, \varepsilon) = \begin{cases} 0 & \text{if } U(\mathbf{x}, \boldsymbol{\beta}, \varepsilon) \leq 0 \\ 1 & \text{if } U(\mathbf{x}, \boldsymbol{\beta}, \varepsilon) > 0 \end{cases} \quad (1.4)$$

The “logit” part of the model’s name refers to the choice of f . The binary logit model assumes f is the logistic distribution:

$$f(\varepsilon) = \frac{e^{-\varepsilon}}{(1 + e^{-\varepsilon})^2} \quad (1.5)$$

Having specified h and f , let’s choose some values for $\mathbf{x}$ and $\boldsymbol{\beta}$ and use simulation to approximate the integral for $p(y|\mathbf{x}, \boldsymbol{\beta})$.³ Let’s pick $\mathbf{x} = (0.5, 2)$ and $\boldsymbol{\beta} = (3, -1)$ such that $\mathbf{x}'\boldsymbol{\beta} = (0.5)(3) + (2)(-1) = 1.5 - 2 = -0.5$. We know from the closed-form solution to this model provided by Train that, with these values of $\mathbf{x}$ and $\boldsymbol{\beta}$, the probability the decision maker takes action is:

$$p(y = 1|\mathbf{x}, \boldsymbol{\beta}) = \frac{e^{\mathbf{x}'\boldsymbol{\beta}}}{1 + e^{\mathbf{x}'\boldsymbol{\beta}}} = \frac{e^{-0.5}}{1 + e^{-0.5}} = 0.3775407 \quad (1.6)$$

We can approximate this integral as before. Below I use the function `rlogis()` to take $R=1000$ draws from the binary logistic distribution, and I approximate the integral with the proportion of times $\mathbf{x}'\boldsymbol{\beta} + \varepsilon$ is greater than the threshold for action (0):

³Only at the early stage of simulating data from a model to better understand the model do we pick values for $\mathbf{x}$ and $\boldsymbol{\beta}$. Typically, $\mathbf{x}$ will part of the data you collect and $\boldsymbol{\beta}$ will be parameters whose values you seek to estimate.

```
set.seed(2345)
R <- 1000

x <- c(0.5, 2)
beta <- c(3, -1)

U <- as.vector(x %*% beta) + rlogis(R)
mean(U > 0)
```

```
[1] 0.364
```

Our simulated value 0.364 approximates the exact value of the integral 0.378.

1.4 Key Learnings

The key learning from this chapter is that with discrete choice models our focus is on the *probability* of the choice outcome y – that is, $p(y|x)$. This choice outcome y results from the joint distribution f of unobserved factors ε and the behavioral model h that relates y to $\mathbf{x}$ (and ε). The probability of the choice outcome $p(y|x)$ can be written in closed form for only very special choices of f and h , but for almost any choice of f and h we can simulate $p(y|x)$, as the two examples in this chapter demonstrate.

2 Drawing from Densities

2.1 Why We Need Draws

2.2 Pseudo-Random Numbers from the Uniform Distribution

2.3 The Inverse Transform Method

$$\begin{aligned} F_X(x) &= \Pr(X \leq x) \\ &= \Pr(T(U) \leq x) \\ &= \Pr(U \leq T^{-1}(x)) \\ &= T^{-1}(x) \end{aligned}$$

2.3.1 Example: Exponential

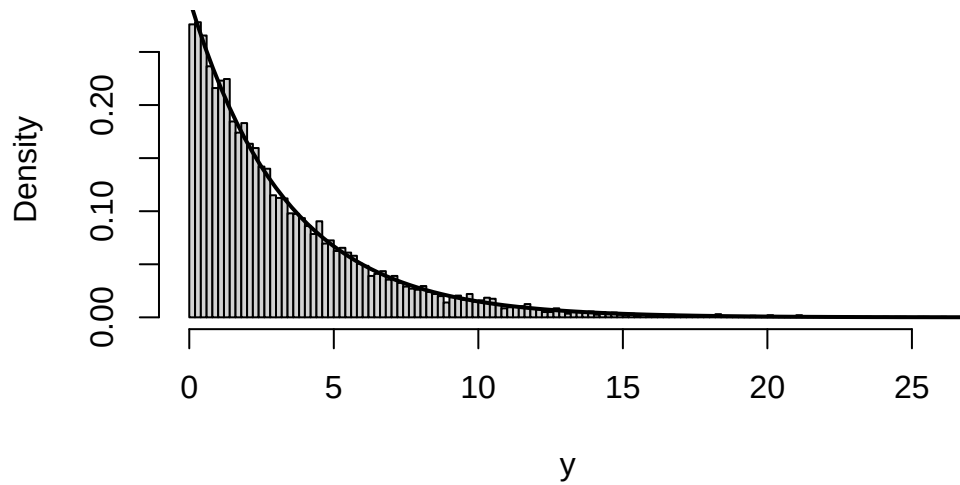
$$F(y, \lambda) = 1 - \exp(-\lambda y) \quad \text{for } y \geq 0$$

$$F^{-1}(u) = -\log(1 - u)/\lambda$$

```
n <- 1e4
lambda <- 0.3
u <- runif(n)
y <- -log(1-u)/lambda
```

```
x <- seq(from=0, to=ceiling(max(y)), length.out=100)
hist(y, freq=F, breaks=100)
lines(x=x, y=dexp(x,lambda), lwd=2)
```

Histogram of y



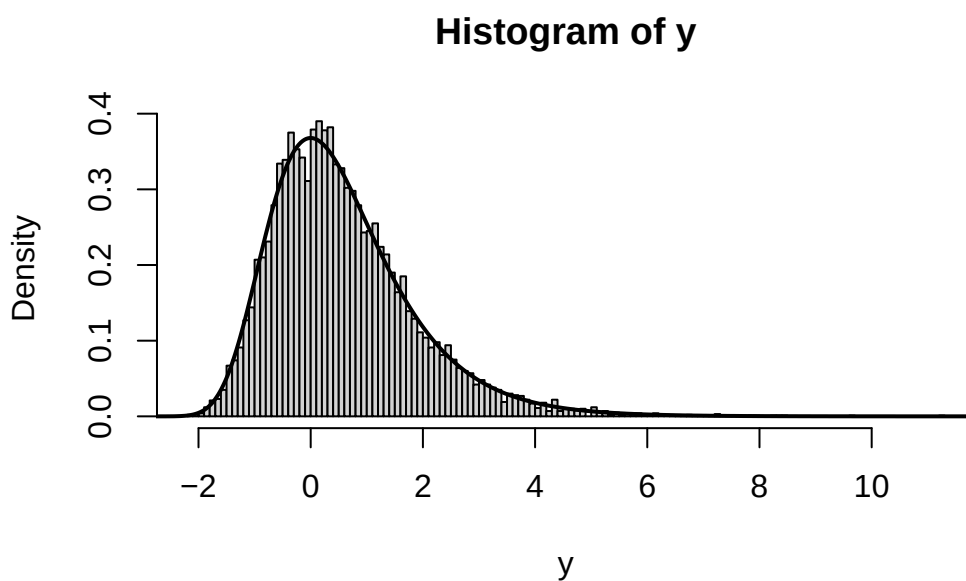
2.3.2 Example: Gumbel

$$F(y) = \exp(-\exp(-y))$$

$$F^{-1}(u) = -\log(-\log(u))$$

```
n <- 1e4  
u <- runif(n)  
y <- -log(-log(u))
```

```
x <- seq(from=-3, to=ceiling(max(y)), length.out=500)  
hist(y, freq=F, breaks=100)  
lines(x=x, y=ordinal::dgumbel(x), lwd=2)
```



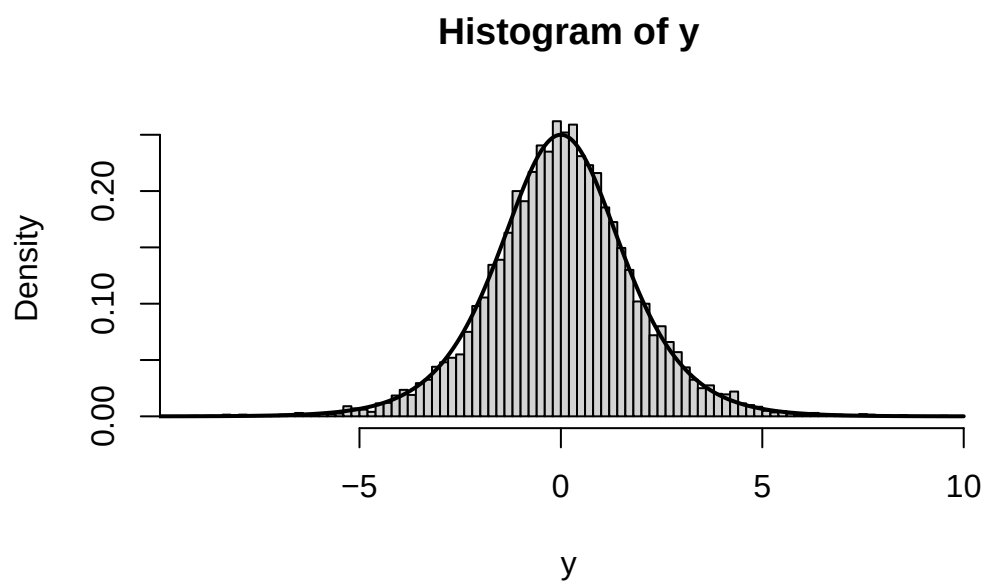
2.3.3 Example: Logistic

$$F(y) = \frac{1}{1 + \exp(-x)}$$

$$F^{-1}(u) = \log(u) - \log(1 - u)$$

```
n <- 1e4  
u <- runif(n)  
y <- log(u) - log(1-u)
```

```
x <- seq(from=floor(min(y)), to=ceiling(max(y)), length.out=500)  
hist(y, freq=F, breaks=100)  
lines(x=x, y=dlogis(x), lwd=2)
```



2.3.4 Example: Normal

```
n <- 1e4  
y <- rnorm(n)  
  
u <- runif(n)  
y <- qnorm(u)
```

2.4 Accept-Reject Methods

2.5 Transforming Draws

2.6 The Multivariate Normal

2.6.1 Example: Correlated Normal Draws

2.7 Truncated Distributions

2.8 Quasi-Random Draws

2.9 Key Learnings

3 Simulating from a Model

3.1 Why Simulate?

3.2 The Anatomy of a Simulation Script

3.3 Example: Simulating Binary Logit Choices

3.4 Example: Simulating Binary Probit Choices

3.5 From One Choice to a Dataset

3.6 Sanity Checks on Simulated Data

3.7 The Simulate-and-Recover Workflow

3.8 Key Learnings

4 Organizing Choice Data

4.1 The Shape of Choice Data

4.2 Wide and Long Formats

4.3 Alternative-Specific and Individual-Specific Variables

4.4 Coding Categorical Attributes

4.5 The Outside Option

4.6 From Data Frame to Likelihood Inputs

4.7 Package Conventions

4.8 Key Learnings

5 From Choice Data to Likelihood

5.1 The Likelihood Function

5.2 Independence and the Product Form

5.3 The Log-Likelihood

5.4 Example: The Binary Logit Log-Likelihood in R

5.5 Maximum Likelihood Estimation

5.6 The Score, the Hessian, and the Information Matrix

5.7 A First Estimator: Grid Search

5.8 Why Grid Search Fails at Scale

5.9 Key Learnings

6 Numerical Optimization

6.1 Climbing a Hill You Cannot See

6.2 Newton-Raphson

6.3 Quasi-Newton Methods: BHHH and BFGS

6.4 Numerical Derivatives

6.5 R's Optimizers

6.6 Standard Errors from the Hessian

6.7 Practical Realities

6.8 End-to-End: Simulate, Estimate, Recover

6.9 Key Learnings

Part II

The MNL Family, by Likelihood

7 The Multinomial Logit Model

7.1 From Two Alternatives to Many

7.2 Random Utility and the Extreme Value Distribution

7.3 The MNL Choice Probability

7.4 Properties and Implications

7.4.1 Independence from Irrelevant Alternatives

7.4.2 Substitution Patterns and Elasticities

7.5 Identification

7.5.1 Only Utility Differences Matter

7.5.2 The Scale of Utility

7.6 Specification Choices

7.7 Simulating MNL Data

7.8 Key Learnings

8 Estimating the MNL by Maximum Likelihood

8.1 The MNL Log-Likelihood

8.2 Coding `loglik_mnl()`

8.3 Numerical Stability: The Log-Sum-Exp Trick

8.4 The Analytic Gradient

8.5 Estimation with `optim()`

8.6 Assessing Fit

8.7 Validation Against `mlogit`

8.8 A Full Simulate-and-Recover Study

8.9 Key Learnings

9 Code Performance

9.1 Why Speed Matters Now

9.2 Measure First

9.3 Vectorizing the MNL Log-Likelihood

9.4 Precompute What Doesn't Change

9.5 Memory Layout and Copies

9.6 When R Isn't Enough

9.7 Key Learnings

10 Relaxing IIA: Nested Logit and GEV Models

10.1 When Proportional Substitution Fails

10.2 The Nested Logit Model

10.2.1 Nests and the Dissimilarity Parameter

10.2.2 Choice Probabilities: Marginal times Conditional

10.3 Simulating Nested Logit Data

10.4 Estimating the Nested Logit

10.5 Validation Against `mlogit`

10.6 The GEV Family

10.7 Key Learnings

11 The Latent Class MNL Model

11.1 Heterogeneity Enters

11.2 The Latent Class Model

11.3 The Likelihood: A Panel Subtlety

11.4 Simulating Latent Class Data

11.5 Direct Maximization

11.6 The EM Algorithm

11.7 How Many Classes?

11.8 Individual-Level Output

11.9 Validation Against `gmnl`

11.10 Key Learnings

12 Maximum Simulated Likelihood

12.1 Choice Probabilities Without Closed Forms

12.2 The Simulated Choice Probability

12.3 The Simulated Log-Likelihood and Its Bias

12.4 Practical Rules of MSL

12.5 Better Draws: Halton and Antithetics

12.6 A First MSL Estimation

12.7 Relatives: MSM and MSS

12.8 Key Learnings

13 The Mixed MNL Model by MSL

13.1 Random Coefficients

13.2 Choosing Mixing Distributions

13.3 The Mixed MNL Likelihood

13.4 Simulating Mixed MNL Data

13.5 Coding the Simulated Log-Likelihood

13.6 Estimation

13.7 Feeling the Strain

13.8 Validation Against `mlogit`, `gmnl`, and `mixl`

13.9 Key Learnings

14 The Limits of Likelihood

14.1 What We Actually Want

14.2 Why Not Estimate Each Person's Beta by MLE?

14.3 Conditioning on Choices: Individual Estimates After MSL

14.4 ... But It Creaks

14.5 The Bayesian Road

14.6 Key Learnings

Part III

The MNL Family, by Bayes

15 Bayesian Estimation of the MNL

15.1 The Bayesian Mindset

15.2 Priors

15.3 The Posterior Has No Closed Form Either

15.4 Summarizing Distributions with Draws

15.5 The Metropolis-Hastings Algorithm

15.5.1 The Idea

15.5.2 A Toy Target First

15.5.3 Why It Works

15.6 M-H for the MNL

15.7 MCMC Diagnostics

15.8 Bayes Meets MLE

15.9 Key Learnings

16 Gibbs Sampling and Hierarchical Models

16.1 Sampling by Blocks

16.1.1 A Toy Example: The Bivariate Normal

16.2 Conjugacy

16.3 The Hierarchical Model

16.3.1 Shrinkage: Borrowing Strength

16.3.2 Gibbs for the Hierarchical Linear Model

16.4 Data Augmentation

16.5 Metropolis-within-Gibbs

16.6 Key Learnings

17 The Hierarchical Bayesian MNL

17.1 The Model

17.2 The Sampler

17.3 Coding the HB-MNL Sampler

17.4 Running It

17.5 The Payoff: Individual-Level Posteriors

17.6 HB and MSL on the Same Data

17.7 Validation Against bayesm

17.8 Practical Notes

17.9 Key Learnings

Part IV

Beyond Logit: The Multinomial Probit

18 The Multinomial Probit Model

18.1 Why Probit?

18.2 The Model

18.3 Identification

18.4 Binary Probit as a Warm-up

18.5 Simulating MNP Data

18.6 The Estimation Problem Ahead

18.7 Key Learnings

19 Estimating the MNP

19.1 Route 1: Simulated Likelihood

19.1.1 The Accept-Reject Simulator and Its Flaws

19.1.2 The GHK Simulator

19.1.3 MSL with GHK

19.2 Route 2: Bayesian Data Augmentation

19.2.1 Latent Utilities as Parameters

19.2.2 The McCulloch-Rossi Gibbs Sampler

19.2.3 Identification Inside the Sampler

19.3 The Two Routes Compared

19.4 Validation Against bayesm

19.5 Key Learnings

20 The Hierarchical Bayesian MNP

20.1 The Model

20.2 The Sampler

20.3 Coding the HB-MNP Sampler

20.4 HB-MNL or HB-MNP?

20.5 Key Learnings

Part V

Using and Extending the Models

21 Post-Estimation: Inference, WTP, and Market Simulation

21.1 Estimates Are Not Answers

21.2 Inference on Functions of Parameters

21.3 Willingness-to-Pay

21.4 Attribute Importance

21.5 Market Simulation

21.6 Elasticities

21.7 Using Individual-Level Estimates

21.8 Key Learnings

22 Extensions: Building Your Own Model

22.1 The Recipe

22.2 Extension 1: The No-Choice Option and Dual Response

22.2.1 Modeling “None”

22.2.2 Dual-Response Designs and Their Likelihoods

22.2.3 What Changes in the Code

22.3 Extension 2: A Budget Constraint

22.3.1 The Model

22.3.2 What Changes in the Code

22.4 Extension 3: Flexible Mixing Distributions

22.4.1 The Logit-Mixed Logit Model

22.4.2 What Changes in the Code

22.5 Where to Go from Here

22.6 Key Learnings

References

- Abramovich, Felix, and Ya'acov Ritov. 2023. *Bayesian Statistics*. CRC press.
- Gelman, Andrew, John B. Carlin, Hal S. Stern, David B. Dunson, Aki Vehtari, and Donald B. Rubin. 2013. *Bayesian Data Analysis*. CRC press.
- Goldberger, Arthur S. 1991. *A Course in Econometrics*. Harvard University Press.
- Hansen, Bruce. 2022a. *Econometrics*. Princeton University Press.
- . 2022b. *Probability & Statistics for Economists*. Princeton University Press.
- Hensher, David A., John M. Rose, and William H. Greene. 2015. *Applied Choice Analysis*. 2nd ed. Cambridge University Press.
- Kennedy, Peter. 2008. *A Guide to Econometrics*. John Wiley & Sons.
- McElreath, Richard. 2018. *Statistical Rethinking: A Bayesian Course with Examples in r and Stan*. Chapman; Hall/CRC.
- Train, Kenneth E. 2009. *Discrete Choice Methods with Simulation*. 2nd ed. Cambridge University Press. <https://eml.berkeley.edu/books/choice2.html>.
- Wickham, Hadley. 2019. *Advanced r*. 2nd ed. Chapman; Hall/CRC.
- Wickham, Hadley, Mine Cetinka-Rundel, and Garrett Grolemond. 2023. *R for Data Science*. 2nd ed. O'Reilly Media.

A Appendix

A.1 R Packages for Discrete Choice

A.2 Notation Glossary

A.3 Common Numerical Errors and Their Symptoms